

```

# =====
# 1. INSTALL LIBRARIES
# =====

!pip install captum shap lime scikit-image grad-cam --quiet

_____ 275.7/275.7 kB 6.3 MB/s eta
0:00:00
etadata (setup.py) ... _____
44.5/44.5 kB 2.6 MB/s eta 0:00:00
ents to build wheel ... etadata (pyproject.toml) ...
_____ 455.2/455.2 kB 15.5 MB/s eta
0:00:00
e (setup.py) ... (pyproject.toml) ...

# =====
# 2. IMPORT LIBRARIES
# =====

import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt

from PIL import Image

import torchvision.transforms as transforms
from torchvision.models import resnet50, ResNet50_Weights

# Captum
from captum.attr import IntegratedGradients
from captum.attr import Occlusion
from captum.attr import GuidedBackprop

# Grad-CAM
from pytorch_grad_cam import GradCAM, GradCAMPlusPlus
from pytorch_grad_cam.utils.image import show_cam_on_image
from pytorch_grad_cam.utils.model_targets import
ClassifierOutputTarget

# LIME
from lime import lime_image
from skimage.segmentation import mark_boundaries

# SHAP
import shap

# =====
# 3. SELECT DEVICE
# =====

```

```

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)

Device: cpu

# =====
# 4. LOAD PRETRAINED RESNET-50
# =====

weights = ResNet50_Weights.DEFAULT

model = resnet50(weights=weights)

model = model.eval().to(device)

Downloading: "https://download.pytorch.org/models/resnet50-1lad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-1lad3fa6.pth

100%|██████████| 97.8M/97.8M [00:01<00:00, 53.8MB/s]

# =====
# 5. DISABLE IN-PLACE ReLU
# =====
# This is important for SHAP.
#
# SHAP DeepExplainer can give:
#
# RuntimeError:
# Output of BackwardHookFunctionBackward is a view
# and is being modified inplace
#
# ResNet-50 contains ReLU(inplace=True).
# We change them to inplace=False.
# =====

def disable_inplace_relu(module):
    for child in module.children():
        if isinstance(child, torch.nn.ReLU):
            child.inplace = False
            disable_inplace_relu(child)

```

```

disable_inplace_relu(model)

print("In-place ReLU disabled")

# =====
# 6. IMAGE PREPROCESSING
# =====

preprocess = weights.transforms()

# ImageNet class labels
imagenet_labels = weights.meta["categories"]

print("ResNet-50 loaded successfully")
In-place ReLU disabled
ResNet-50 loaded successfully

# =====
# 7. UPLOAD YOUR PHOTO
# =====

from google.colab import files

print("=====")
print("UPLOAD YOUR PHOTO")
print("=====")

uploaded = files.upload()

# Get uploaded file name
image_path = list(uploaded.keys())[0]

# Open image
img_pil = Image.open(
    image_path
).convert("RGB")

# Display original image
plt.figure(figsize=(6, 5))

plt.imshow(img_pil)

plt.title("Original Image")

plt.axis("off")

```

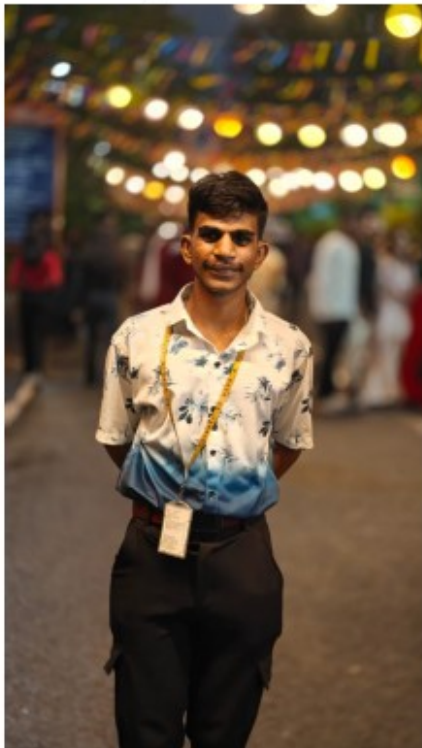
```
plt.show()
```

```
=====
UPLOAD YOUR PHOTO
=====
```

```
<IPython.core.display.HTML object>
```

```
Saving IMG-20260429-WA0016.jpg to IMG-20260429-WA0016 (1).jpg
```

Original Image



```
# =====
# 8. PREPARE IMAGE
# =====

def prepare_image(img):
    tensor = preprocess(img)

    # Add batch dimension
    tensor = tensor.unsqueeze(0)

    # Move to CPU/GPU
    tensor = tensor.to(device)
```

```

        return tensor

input_tensor = prepare_image(img_pil)

print("Image prepared successfully")

print(
    "Input tensor shape:",
    input_tensor.shape
)

Image prepared successfully
Input tensor shape: torch.Size([1, 3, 224, 224])

# =====
# 9. PREDICTION
# =====

def predict(input_tensor):
    model.eval()

    with torch.no_grad():
        logits = model(input_tensor)

        probs = torch.softmax(
            logits,
            dim=1
        )

    return probs, logits

# Get prediction
probs, logits = predict(input_tensor)

# Find highest probability
top_prob, top_idx = probs.max(dim=1)

# Get target class ID
target = top_idx.item()

# Get class name
pred_label = imagenet_labels[target]

```

```

print()
print("=====")
print("RESNET-50 PREDICTION")
print("=====")

print(
    "Predicted class :",
    pred_label
)

print(
    "Class ID :",
    target
)

print(
    "Probability :",
    round(top_prob.item(), 4)
)

print(
    "Confidence :",
    round(top_prob.item() * 100, 2),
    "%"
)

=====
RESNET-50 PREDICTION
=====
Predicted class : sunglass
Class ID : 836
Probability : 0.0976
Confidence : 9.76 %

# =====
# 10. OCCLUSION
# =====

print()
print("Running Occlusion...")

occlusion = Occlusion(model)

attributions_occ = occlusion.attribute(
    input_tensor,
    strides=(1, 8, 8),

```

```

        target=target,
        sliding_window_shapes=(1, 15, 15),
        baselines=0
    )

# Convert tensor to NumPy
    attr_occ = (
        attributions_occ
        .squeeze()
        .detach()
        .cpu()
        .numpy()
    )

# Sum RGB channels
    attr_occ = attr_occ.sum(axis=0)

# Normalize
    attr_occ_norm = (
        attr_occ - attr_occ.min()
    ) / (
        attr_occ.max()
        - attr_occ.min()
        + 1e-8
    )

# Display
    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.imshow(img_pil)

```

```
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(img_pil)
plt.imshow(
    attr_occ_norm,
    cmap="jet",
    alpha=0.5
)
plt.title("Occlusion Sensitivity")
plt.axis("off")

plt.tight_layout()
plt.show()

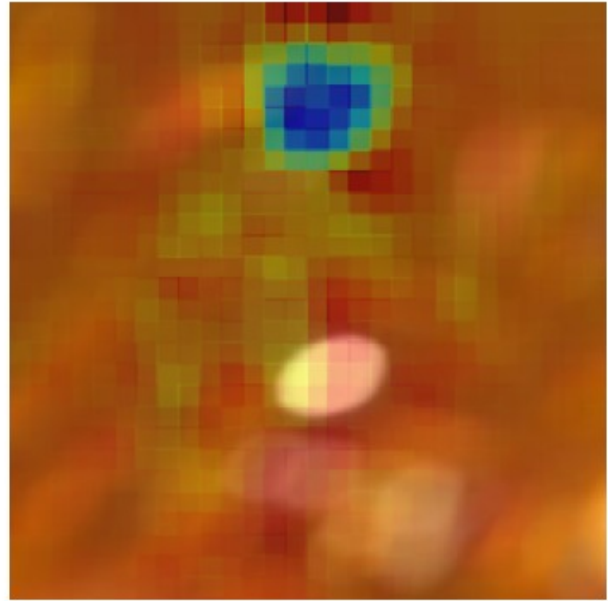
print("Occlusion completed.")
```

Running Occlusion...

Original



Occlusion Sensitivity



Occlusion completed.

```
# =====  
# 11. INTEGRATED GRADIENTS  
# =====  
  
print()  
print("Running Integrated Gradients...")  
  
ig = IntegratedGradients(model)  
  
# Create black/zero baseline  
baseline = torch.zeros_like(  
    input_tensor  
) .to(device)  
  
attributions_ig = ig.attribute(  
    input_tensor,  
    baselines=baseline,  
    target=target,  
    n_steps=50  
)
```

```
# Convert to NumPy
attr_ig = (
    attributions_ig
    .squeeze()
    .detach()
    .cpu()
    .numpy()
)

# Sum RGB channels
attr_ig = attr_ig.sum(axis=0)

# Normalize
attr_ig_norm = (
    attr_ig - attr_ig.min()
) / (
    attr_ig.max()
    - attr_ig.min()
    + 1e-8
)

# Display
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(img_pil)
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(img_pil)
```

```
plt.imshow(
    attr_ig_norm,
    cmap="hot",
    alpha=0.5
)
plt.title("Integrated Gradients")
plt.axis("off")

plt.tight_layout()
plt.show()

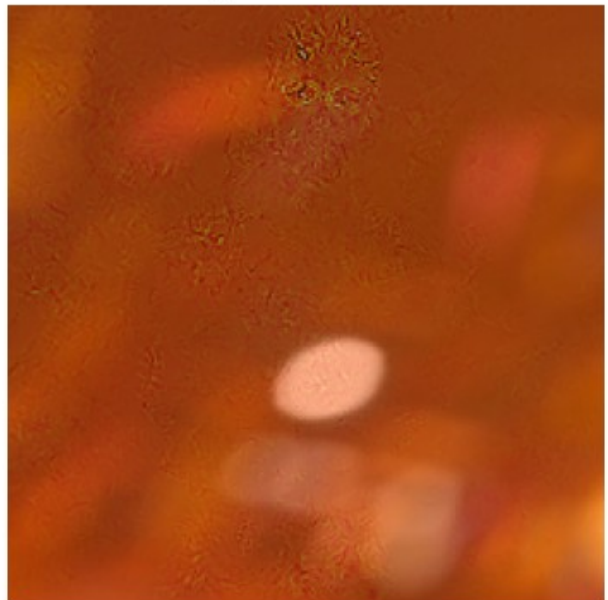
print("Integrated Gradients completed.")
```

Running Integrated Gradients...

Original



Integrated Gradients



Integrated Gradients completed.

```
# =====
# 12. GUIDED BACKPROPAGATION
```

```
# =====  
  
print()  
print("Running Guided Backpropagation...")  
  
gbp = GuidedBackprop(model)  
  
# Create a fresh tensor  
gbp_input = input_tensor.detach().clone()  
gbp_input.requires_grad_(True)  
  
attributions_gbp = gbp.attribute(  
    gbp_input,  
    target=target  
)  
  
# Convert to NumPy  
attr_gbp = (  
    attributions_gbp  
    .squeeze()  
    .detach()  
    .cpu()  
    .numpy()  
)  
  
# Absolute values  
attr_gbp = np.abs(attr_gbp)  
  
# Sum RGB channels  
attr_gbp = attr_gbp.sum(axis=0)  
  
# Normalize  
attr_gbp_norm = (  
    attr_gbp - attr_gbp.min()
```

```

) / (
    attr_gbp.max()
    - attr_gbp.min()
    + 1e-8
)

# Display
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(img_pil)
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(
    attr_gbp_norm,
    cmap="gray"
)
plt.title("Guided Backpropagation")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Guided Backpropagation completed.")

# =====
# 13. GRAD-CAM
# =====

print()
print("Running Grad-CAM...")

```

```
model.eval()

# Select final convolutional layer
target_layers = [
    model.layer4[-1]
]

# Create Grad-CAM
cam = GradCAM(
    model=model,
    target_layers=target_layers
)

# Target class
targets = [
    ClassifierOutputTarget(target)
]

# Generate CAM
grayscale_cam = cam(
    input_tensor=input_tensor,
    targets=targets
)[0]

# CAM dimensions
H_cam, W_cam = grayscale_cam.shape

# Resize image
img_resized = img_pil.resize(
    (W_cam, H_cam)
)
```

```
# Convert to RGB NumPy [0,1]
rgb_img = (
    np.array(img_resized)
    .astype(np.float32)
    / 255.0
)

# Create overlay
visualization = show_cam_on_image(
    rgb_img,
    grayscale_cam,
    use_rgb=True
)

# Display
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(img_pil)
plt.title(
    f"Original\n{pred_label}"
)
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(visualization)
plt.title("Grad-CAM")
plt.axis("off")

plt.tight_layout()
```

```
plt.show()
```

```
print("Grad-CAM completed.")
```

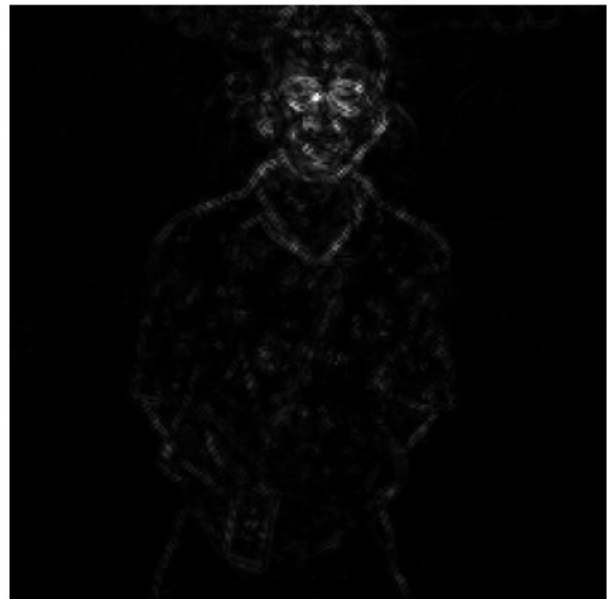
Running Guided Backpropagation...

```
/usr/local/lib/python3.13/dist-packages/captum/attr/_core/  
guided_backprop_deconvnet.py:66: UserWarning: Setting backward hooks  
on ReLU activations.The hooks will be removed after the attribution is  
finished  
warnings.warn(
```

Original



Guided Backpropagation



Guided Backpropagation completed.

Running Grad-CAM...

Original
sunglass



Grad-CAM



Grad-CAM completed.

```
# =====  
# 14. GRAD-CAM++  
# =====  
  
print()  
print("Running Grad-CAM++...")  
  
target_layers = [  
    model.layer4[-1]  
]  
  
# Create Grad-CAM++  
cam_pp = GradCAMPlusPlus(  
    model=model,  
    target_layers=target_layers  
)  
  
targets = [  
    ClassifierOutputTarget(target)
```

```
]

# Generate CAM++
grayscale_cam_pp = cam_pp(
    input_tensor=input_tensor,
    targets=targets
)[0]

# CAM dimensions
H_cam, W_cam = grayscale_cam_pp.shape

# Resize image
img_resized_pp = img_pil.resize(
    (W_cam, H_cam)
)

# Convert to RGB NumPy [0,1]
rgb_img_pp = (
    np.array(img_resized_pp)
    .astype(np.float32)
    / 255.0
)

# Create overlay
visualization_pp = show_cam_on_image(
    rgb_img_pp,
    grayscale_cam_pp,
    use_rgb=True
)

# Display
plt.figure(figsize=(10, 4))
```

```
plt.subplot(1, 2, 1)
plt.imshow(img_pil)
plt.title(
    f"Original\n{pred_label}"
)
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(visualization_pp)
plt.title("Grad-CAM++")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Grad-CAM++ completed.")
```

Running Grad-CAM++...

Original
sunglass



Grad-CAM++



Grad-CAM++ completed.

```
# =====  
# 15. LIME  
# =====  
  
print()  
print("Running LIME...")  
  
# Function required by LIME  
def batch_predict(images):  
    model.eval()  
    batch_tensors = []  
  
    for img in images:  
        # LIME gives H x W x 3 values in [0,1]  
        pil_img = Image.fromarray(  
            (img * 255).astype(np.uint8)  
        )  
  
        # Apply ResNet preprocessing  
        tensor = preprocess(pil_img)
```

```

        batch_tensors.append(tensor)

    # Create batch
    batch = torch.stack(
        batch_tensors
    ).to(device)

    # Prediction
    with torch.no_grad():
        logits = model(batch)
        probs = torch.softmax(
            logits,
            dim=1
        )

    return probs.cpu().numpy()

# Convert original image to HWC [0,1]
img_lime = (
    np.array(img_pil)
    .astype(np.float32)
    / 255.0
)

# Create LIME explainer
explainer = lime_image.LimeImageExplainer()

# Generate explanation
explanation = explainer.explain_instance(
    img_lime,
    batch_predict,

```

```

        top_labels=5,
        hide_color=0,
        num_samples=100
    )

    # Explain predicted class
    lime_label = target

    temp, mask = (
        explanation.get_image_and_mask(
            lime_label,
            positive_only=True,
            num_features=5,
            hide_rest=False
        )
    )

    # Display LIME
    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.imshow(img_pil)
    plt.title(
        f"Original\n{pred_label}"
    )
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(

```

```
mark_boundaries(  
    temp / 255.0,  
    mask  
)  
)  
plt.title(  
    "LIME - Important Superpixels"  
)  
plt.axis("off")  
  
plt.tight_layout()  
plt.show()  
  
print("LIME completed.")  
  
Running LIME...  
{  
    "model_id": "9ebe325261a449cbb87d50823f5001c3",  
    "version_major": 2,  
    "version_minor": 0  
}
```

Original
sunglass



LIME - Important Superpixels



LIME completed.

```
# =====  
# 16. SHAP IMAGE EXPLANATION  
# =====  
  
import shap  
import torch  
import numpy as np  
import matplotlib.pyplot as plt  
  
from torchvision import transforms  
  
print()  
print("=" * 60)  
print("Running SHAP...")  
print("=" * 60)  
  
# =====  
# 1. Prepare model  
# =====  
  
model = model.to(device)  
model.eval()
```

```

print("Model ready.")

# =====
# 2. Prepare image
# =====

# ResNet expects 224 x 224
SHAP_SIZE = 224

img_shap_pil = img_pil.convert("RGB").resize(
    (SHAP_SIZE, SHAP_SIZE)
)

# Convert to numpy H x W x C
img_shap_np = np.asarray(
    img_shap_pil,
    dtype=np.float32
) / 255.0

print(
    "SHAP image shape:",
    img_shap_np.shape
)

# =====
# 3. Determine target class
# =====

# Create model input using your existing preprocess
shap_input = preprocess(
    img_shap_pil
).unsqueeze(0).to(device)

with torch.no_grad():
    output = model(
        shap_input
    )

    target_class = int(
        torch.argmax(
            output,
            dim=1
        ).item()
    )

print(

```

```

        "Target class:",
        target_class
    )

# =====
# 4. Get predicted label
# =====

try:
    display_label = pred_label
except NameError:
    display_label = f"Class {target_class}"

# =====
# 5. Create prediction function for SHAP
#
# SHAP will provide images as:
#
#     N x H x W x 3
#
# with values in [0, 1].
#
# We convert them to PyTorch tensors using your existing
# preprocessing pipeline.
# =====

def shap_predict(images):
    # Make sure input is numpy
    images = np.asarray(
        images,
        dtype=np.float32
    )

    # Handle single image
    if images.ndim == 3:
        images = images[np.newaxis, ...]

    predictions = []

    for image in images:
        # -----
        # Convert numpy image to PIL
        # -----

```

```

image_uint8 = np.clip(
    image * 255.0,
    0,
    255
).astype(
    np.uint8
)

image_pil = transforms.ToPILImage()(
    image_uint8
)

# -----
# Apply your existing preprocessing
# -----

tensor = preprocess(
    image_pil
)

tensor = tensor.unsqueeze(
    0
).to(device)

# -----
# Forward pass
# -----

with torch.no_grad():
    logits = model(
        tensor
    )

# -----
# Return target-class logit
#
# SHAP explains this specific class.
# -----

target_logit = logits[
    0,
    target_class
]

predictions.append(
    float(
        target_logit.detach().cpu()
    )
)

```

```

    )

    return np.asarray(
        predictions,
        dtype=np.float64
    )

# =====
# 6. Test prediction function
# =====

print("Testing SHAP prediction function...")

test_prediction = shap_predict(
    img_shap_np[np.newaxis, ...]
)

print(
    "Target-class output:",
    test_prediction
)

# =====
# 7. Create SHAP image masker
#
# Unlike GradientExplainer, this does NOT register backward
# hooks inside ResNet.
#
# Therefore it avoids:
#
#     BackwardHookFunctionBackward
#
# and the:
#
#     512 vs 2048
#
# error from ResNet-50 bottleneck layers.
# =====

print("Creating SHAP image masker...")

masker = shap.maskers.Image(
    "blur(32,32)",
    (
        SHAP_SIZE,

```

```

        SHAP_SIZE,
        3
    )
)

print(
    "SHAP image masker created."
)

# =====
# 8. Create model-agnostic SHAP explainer
# =====

print(
    "Creating SHAP PartitionExplainer..."
)

explainer_shap = shap.Explainer(
    shap_predict,
    masker,
    algorithm="partition"
)

print(
    "SHAP PartitionExplainer created successfully."
)

# =====
# 9. Calculate SHAP values
#
# max_evals controls computation time.
#
# 500 is a reasonable starting value for a 224x224 image.
# Increase to 1000 or 2000 for a more detailed explanation.
# =====

print()
print(
    "Calculating SHAP values..."
)
print(
    "This may take some time because this is a "
    "model-agnostic image explanation."
)

```

```

shap_result = explainer_shap(
    img_shap_np[np.newaxis, ...],
    max_evals=500,
    batch_size=8
)

print(
    "SHAP values calculated successfully."
)

# =====
# 10. Extract SHAP values
# =====

sv = shap_result.values

print(
    "Raw SHAP values shape:",
    sv.shape
)

# =====
# 11. Remove batch dimension
# =====

if sv.ndim == 4:

    # Expected:
    #
    # 1 x H x W x 3

    sv = sv[0]

# =====
# 12. Convert RGB SHAP values to 2D importance map
# =====

if sv.ndim == 3:

    if sv.shape[-1] == 3:

        # H x W x C
        #
        # Use absolute contribution magnitude
        # for the importance visualization.

```

```

        sv_agg = np.mean(
            np.abs(sv),
            axis=-1
        )

    elif sv.shape[0] == 3:

        # C x H x W
        sv_agg = np.mean(
            np.abs(sv),
            axis=0
        )

    else:

        raise RuntimeError(
            "Unexpected SHAP channel dimension: "
            f"{sv.shape}"
        )

else:

    raise RuntimeError(
        "Unexpected SHAP output shape: "
        f"{sv.shape}"
    )

print(
    "2D SHAP map shape:",
    sv_agg.shape
)

# =====
# 13. Normalize SHAP importance map
# =====

max_abs = np.max(
    np.abs(sv_agg)
)

if max_abs > 1e-12:

    sv_norm = (
        sv_agg / max_abs
    )

else:

```

```

    sv_norm = np.zeros_like(
        sv_agg
    )

# =====
# 14. Create signed SHAP map
#
# This preserves positive/negative contributions.
# =====

if sv.ndim == 3:
    if sv.shape[-1] == 3:
        sv_signed = np.mean(
            sv,
            axis=-1
        )

    elif sv.shape[0] == 3:
        sv_signed = np.mean(
            sv,
            axis=0
        )

    else:
        sv_signed = sv_agg
else:
    sv_signed = sv_agg

signed_max = np.max(
    np.abs(sv_signed)
)

if signed_max > 1e-12:
    sv_signed_norm = (
        sv_signed / signed_max
    )
else:
    sv_signed_norm = np.zeros_like(
        sv_signed
    )

```

```

    )

# =====
# 15. Prepare RGB image for visualization
# =====

rgb_img_shap = np.asarray(
    img_shap_pil,
    dtype=np.float32
) / 255.0

# =====
# 16. Calculate positive/negative regions
# =====

positive_map = np.maximum(
    sv_signed_norm,
    0
)

negative_map = np.minimum(
    sv_signed_norm,
    0
)

# =====
# 17. Display SHAP results
# =====

plt.figure(
    figsize=(20, 10)
)

# -----
# Original image
# -----

plt.subplot(
    2,
    3,
    1
)

plt.imshow(
    rgb_img_shap
)

```

```
plt.title(  
    f"Original\n{display_label}"  
)
```

```
plt.axis("off")
```

```
# -----  
# SHAP importance  
# -----
```

```
plt.subplot(  
    2,  
    3,  
    2  
)
```

```
plt.imshow(  
    sv_norm,  
    cmap="hot",  
    vmin=0,  
    vmax=1  
)
```

```
plt.colorbar(  
    fraction=0.046,  
    pad=0.04  
)
```

```
plt.title(  
    "SHAP Importance"  
)
```

```
plt.axis("off")
```

```
# -----  
# Signed SHAP map  
# -----
```

```
plt.subplot(  
    2,  
    3,  
    3  
)
```

```
plt.imshow(  
    sv_signed_norm,  
    cmap="bwr",  
    vmin=-1,  
    vmax=1
```

```

)

plt.colorbar(
    fraction=0.046,
    pad=0.04
)

plt.title(
    "Signed SHAP Map"
)

plt.axis("off")

# -----
# SHAP overlay
# -----

plt.subplot(
    2,
    3,
    4
)

plt.imshow(
    rgb_img_shap
)

plt.imshow(
    sv_norm,
    cmap="hot",
    alpha=0.50,
    vmin=0,
    vmax=1
)

plt.title(
    "SHAP Importance Overlay"
)

plt.axis("off")

# -----
# Positive SHAP
# -----

plt.subplot(
    2,
    3,

```

```

    5
)
plt.imshow(
    rgb_img_shap
)

plt.imshow(
    positive_map,
    cmap="Reds",
    alpha=0.60,
    vmin=0,
    vmax=1
)

plt.title(
    "Positive Contribution"
)

plt.axis("off")

# -----
# Negative SHAP
# -----

plt.subplot(
    2,
    3,
    6
)

plt.imshow(
    rgb_img_shap
)

plt.imshow(
    negative_map,
    cmap="Blues",
    alpha=0.60,
    vmin=-1,
    vmax=0
)

plt.title(
    "Negative Contribution"
)

plt.axis("off")

```

```

plt.tight_layout()

plt.show()

# =====
# 18. Print summary
# =====

print()
print("=" * 60)
print("SHAP completed successfully!")
print("=" * 60)

print(
    "Target class:",
    target_class
)

print(
    "Predicted label:",
    display_label
)

print(
    "SHAP map shape:",
    sv_norm.shape
)

print(
    "Maximum SHAP importance:",
    float(np.max(sv_norm))
)

print(
    "Minimum signed SHAP value:",
    float(np.min(sv_signed_norm))
)

print(
    "Maximum signed SHAP value:",
    float(np.max(sv_signed_norm))
)

=====
Running SHAP...
=====
Model ready.
SHAP image shape: (224, 224, 3)

```

```

Target class: 836
Testing SHAP prediction function...
Target-class output: [5.01019049]
Creating SHAP image masker...
SHAP image masker created.
Creating SHAP PartitionExplainer...
SHAP PartitionExplainer created successfully.

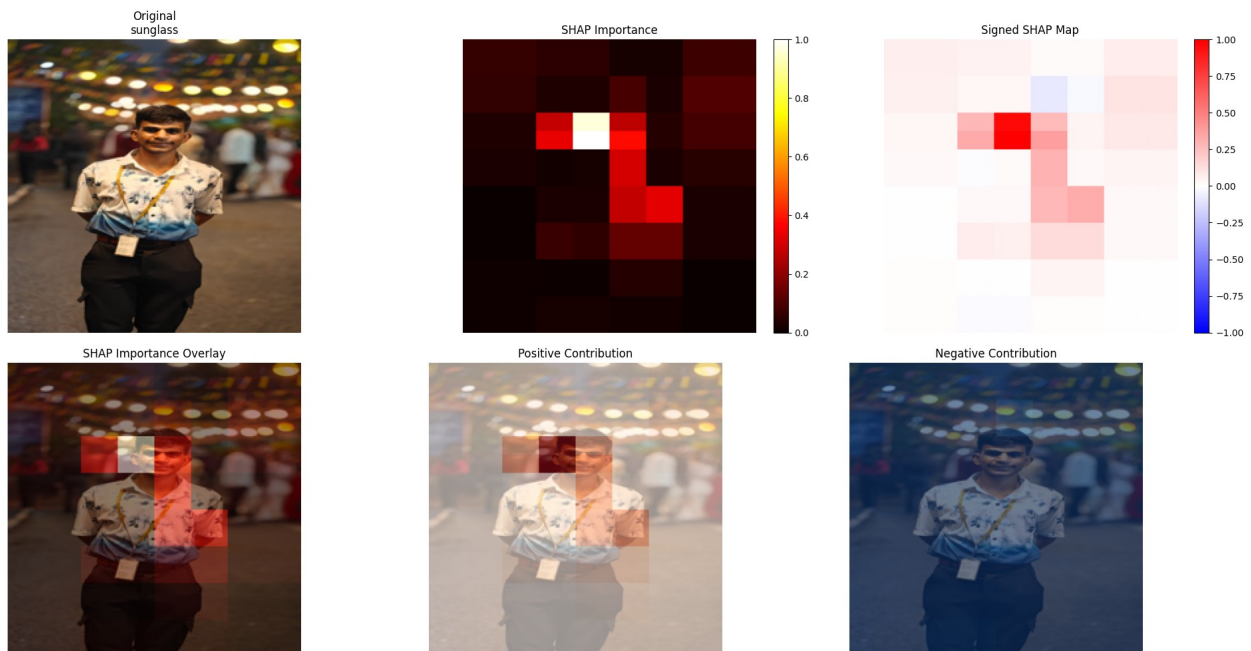
Calculating SHAP values...
This may take some time because this is a model-agnostic image
explanation.

{"model_id":"e6a480374dd143d88d7c1197cbc0b835","version_major":2,"version_minor":0}

PartitionExplainer explainer: 2it [02:04, 124.24s/it]

SHAP values calculated successfully.
Raw SHAP values shape: (1, 224, 224, 3)
2D SHAP map shape: (224, 224)

```



```

=====
SHAP completed successfully!
=====

```

```

Target class: 836
Predicted label: sunglass
SHAP map shape: (224, 224)
Maximum SHAP importance: 1.0

```

Minimum signed SHAP value: -0.09372322394282126
Maximum signed SHAP value: 1.0

```
# =====  
# 17. FINAL SUMMARY  
# =====
```

```
print()  
print("=" * 55)  
  
print(" XAI ANALYSIS COMPLETED")  
print("=" * 55)  
  
print()  
print(  
    "Predicted Class :",  
    pred_label  
)  
  
print(  
    "Class ID :",  
    target  
)  
  
print(  
    "Confidence :",  
    round(  
        top_prob.item() * 100,  
        2  
    ),  
    "%"  
)  
  
print()  
print("XAI Methods Applied:")  
print("1. Occlusion")  
print("2. Integrated Gradients")  
print("3. Guided Backpropagation")  
print("4. Grad-CAM")
```

```

print("5. Grad-CAM++")
print("6. LIME")
print("7. SHAP")

print()

print("All XAI analysis completed successfully!")

```

```

=====
XAI ANALYSIS COMPLETED
=====

```

```

Predicted Class : sunglass
Class ID : 836
Confidence : 9.76 %

```

```

XAI Methods Applied:

```

1. Occlusion
2. Integrated Gradients
3. Guided Backpropagation
4. Grad-CAM
5. Grad-CAM++
6. LIME
7. SHAP

```

All XAI analysis completed successfully!

```

```

probs = torch.softmax(output, dim=1)
top_probs, top_indices = torch.topk(probs, 5)

```

```

for prob, idx in zip(top_probs[0], top_indices[0]):
    print(idx.item(), prob.item() * 100)

```

```

836 6.841307133436203
610 5.926182493567467
841 4.119710624217987
837 4.011194407939911
902 3.133603557944298

```

```

probs = torch.softmax(output, dim=1)
top_probs, top_indices = torch.topk(probs, 5)

```

```

for rank, (prob, idx) in enumerate(zip(top_probs[0], top_indices[0]),
1):
    print(f"{rank}. Class ID: {idx.item()} | Probability:
{prob.item()*100:.2f}%")

```

```
1. Class ID: 836 | Probability: 6.84%
2. Class ID: 610 | Probability: 5.93%
3. Class ID: 841 | Probability: 4.12%
4. Class ID: 837 | Probability: 4.01%
5. Class ID: 902 | Probability: 3.13%
```

```
predicted_class_id = torch.argmax(probs, dim=1).item()
```

```
print("Predicted Class ID:", predicted_class_id)
```

```
print("Probability:", probs[0, predicted_class_id].item() * 100)
```

```
Predicted Class ID: 836
```

```
Probability: 6.841307133436203
```

```
from torchvision.models import ResNet50_Weights
```

```
weights = ResNet50_Weights.DEFAULT
```

```
categories = weights.meta["categories"]
```

```
predicted_class_id = torch.argmax(probs, dim=1).item()
```

```
print("Class ID:", predicted_class_id)
```

```
print("Class Name:", categories[predicted_class_id])
```

```
print("Probability:", probs[0, predicted_class_id].item() * 100)
```

```
Class ID: 836
```

```
Class Name: sunglass
```

```
Probability: 6.841307133436203
```